

Danmarks
Tekniske
Universitet



22053 PRINCIPLES OF BRAIN-COMPUTER INTERFACES

EEG-based Brain-Computer Interface and Classification of a Motor Imagery task

Francesco Balducci	Lars Truttenbach
DTU	DTU
s250200@dtu.dk	s254604@dtu.dk

July 13, 2026

Abstract

This report addresses the classification of motor imagery (MI) tasks through EEG-based brain-computer interfaces. Specifically, it pursues the design of a signal-processing pipeline that combines static and spatial filtering, time–frequency feature extraction and neural network–based classification. Preprocessing steps remove noise and confine the signal to the mu and beta bands, while spatial filters such as PCA and CSP generate low-dimensional representations that preserve the relevant information. The Short-Time Fourier Transform captures the time-varying nature of the spectral content, thus yielding compact features that can be effectively exploited by machine learning models. Feedforward and convolutional neural networks are explored and compared in terms of their performance and robustness. Results show that CSP combined with STFT and an FNN attains high classification accuracy (around 75%) while keeping the pipeline interpretable and computationally light. The report outlines the strengths and weaknesses of both feature-based and deep-learning approaches for MI EEG data.

Contents

1	Introduction	1
1.1	Background	1
1.2	Research Gap	1
1.3	Objectives	2
2	Literature Review	2
2.1	Spatial Filtering	2
2.2	Spectral Features	2
2.3	Classification Approaches	3
3	Methods	3
3.1	Static filtering	4
3.1.1	Notch filter	4
3.1.2	Chebyshev Type II bandpass filter	4
3.2	Spatial filtering	4
3.2.1	Common Spatial Pattern	4
3.2.2	Principal Component Analysis	5
3.3	Feature extraction	5
3.3.1	Short-Time Fourier Transform	5
3.4	Classification	6
3.4.1	Feedforward Neural Network	6
3.4.2	Convolutional Neural Network	7
3.4.3	Cross-validation	7
3.4.4	Evaluation	7
4	Results	8
4.1	Static filtering	8
4.2	STFT	8
4.3	PCA	9
4.4	CSP	10
4.5	Neural networks (FNN and CNN)	11
4.6	Dimensionality across the pipeline	12
5	Discussion	13
5.1	Pre-processing	13
5.2	Feature Extraction	13
5.3	Classification	13
6	Conclusions	14
A	Appendix	i
A.1	CNN architecture	i

1 Introduction

The human brain is an extremely complex organ. It regulates behavior, cognitive processes, motor function, and sensory perception. Understanding and relating it to neural activity has long been a major goal of biomedical engineering and neuroscience. A promising method for establishing direct communication between the brain and external devices is Brain-Computer Interfaces (BCIs), which translate neural activity into control signals for computers, prosthetics, or other assistive technologies [1].

Electroencephalography (EEG)-based BCI paradigms are particularly attractive since they are non-invasive, affordable, and offer high temporal resolution [2]. An important use of EEG-based BCIs is the classification of motor imagery (MI) tasks. Motor imagery is the mental replication of a movement without its actual execution. For example, the EEG shows unique patterns when one imagines moving the left or right wrist, mainly in the sensorimotor rhythm (SMR) over the motor cortex [3]. Since BCI allows users to control devices solely with their thoughts, its effectiveness depends on recognizing and classifying these MI patterns. In situations involving human-computer interaction and therapeutic rehabilitation, such as recovery of motor function after stroke, MI-based BCIs have demonstrated considerable promise [2], [4].

1.1 Background

EEG measures electrical activity at the scalp, allowing to detect changes in brain dynamics. However, EEG signals are weak, non-stationary, and sensitive to noise and artifacts, which makes reliable signal processing challenging [5], [6].

EEG-based BCIs can be divided into evoked and spontaneous systems [6]. Evoked BCIs rely on external stimuli like visual flickers or target events, to elicit predictable brain responses (e.g., SSVEP or P300). These systems often achieve high accuracy

with limited training but require continuous attention to stimuli, which can be mentally demanding [7]. In contrast, spontaneous BCIs do not depend on external stimulation. Instead, they interpret internally generated brain activity, such as that produced during MI tasks [8].

Motor imagery BCIs use changes in SMRs, which are brain waves in EEG signals that appear when a person plans or imagines a movement. EEG signals are commonly analyzed in frequency bands, which reflect different aspects of neural activity. The bands can be split in delta (<4 Hz), theta (4–7 Hz), alpha (8–12 Hz), beta (13–30 Hz), and gamma (>30 Hz), with the alpha and beta bands being particularly relevant for MI [9]. During imagined movement, power in these bands typically decreases over the sensorimotor cortex, a phenomenon known as event-related desynchronization (ERD), while power increases are referred to as event-related synchronization (ERS). These patterns differ between imagined movements, such as left and right wrist imagery, and can therefore be used for classification [3], [10].

1.2 Research Gap

Despite much research on MI-based BCIs [7], [8], [9], several challenges continue to limit their performance and applicability [6]. A key issue is the high variability of EEG signals, both across subjects and across recording sessions, which makes it difficult to extract stable and generalizable features [11]. In addition, EEG recordings exhibit a low signal-to-noise ratio (SNR) and are highly susceptible to artifacts, complicating reliable detection of motor imagery-related neural patterns [6], [7].

Another important gap concerns the effective integration of spatial and spectral information. While motor imagery is known to modulate sensorimotor rhythms in the mu and beta frequency bands, the optimal way to combine spatial filtering and frequency-domain features remains an open question [10], [12]. Finally, many MI-BCI approaches prioritize classification accuracy without sufficiently addressing the

interpretability and validation of feature extraction methods, limiting insight into whether extracted features reflect meaningful neurophysiological processes [6], [13].

These challenges highlight the need for robust, and interpretable signal processing pipelines for motor imagery EEG analysis.

1.3 Objectives

This report focuses on the classification of offline MI-EEG data to develop a framework for interpreting user intent. The primary aim is to identify the most effective methods for integrating signal processing and classification algorithms to generate meaningful output from EEG-based motor imagery. To achieve this, the commonly used BCI pipeline was considered. After data preprocessing, relevant features were extracted and subsequently classified. The main objectives to be addressed can be stated as follows:

1. Preprocess the given datasets by denoising them in a manner such that the most important information about the signal is preserved. Apply static and spatial filters.
2. Test the different classifiers to determine which will provide the best applicability and accuracy.
3. Distinguish between left- and right-wrist movements.
4. Make an overall assessment of the pipeline interpretability and consistency, verifying that each stage of processing respects the established principles underlying signal-processing and/or neurophysiology.

2 Literature Review

2.1 Spatial Filtering

A key group of feature extraction methods MI electroencephalography (EEG) is spatial filtering. This process transforms multichannel EEG into a form

that improves class separability. The Common Spatial Pattern (CSP) algorithm is one of the most well-known spatial filtering methods. CSP identifies linear combinations of EEG channels, maximizing variance for one class and minimizing it for the other. This makes CSP effective for distinguishing between left and right motor imagery tasks. Researchers have frequently combined CSP with mu and beta band features to improve accuracy in MI tasks [8]. Some extensions of CSP, like transformed CSP (tCSP), incorporate additional spectral methods to further enhance discriminative power beyond the traditional approach [10]. Other research merges CSP with channel selection or entropy-based techniques. This helps eliminate noisy or redundant channels, reducing dimensionality while keeping important spatial information [11].

In addition to standard CSP, various studies have looked at filter bank CSP (FBCSP). This method breaks down EEG into multiple overlapping frequency bands and applies CSP independently to each band before selecting features and classifying. This approach improves robustness against individual differences in optimal frequency content and often leads to better performance [12]. Furthermore, information-theoretic versions of CSP have been suggested. These versions choose the most informative features rather than depending only on variance criteria, connecting CSP with measures of mutual information [13]. These developments show the ongoing effort to refine spatial filtering for improved MI discrimination.

2.2 Spectral Features

Spectral features, especially mu (8-13 Hz) and beta (13-30 Hz) band power, are well recognized as useful for MI because motor imagery affects sensorimotor rhythms in these bands. Techniques to compute spectral power include Fourier or Welch methods to derive power spectral density (PSD). This PSD can be used on its own or combined with spatial filters like CSP. Classic studies have shown that combining

CSP with spectral band power features can achieve high classification accuracies for MI tasks [13]. These methods take advantage of known neurophysiological patterns, but they often need careful preprocessing and adjustments for each subject.

2.3 Classification Approaches

Once features are extracted using spatial and spectral methods, supervised classifiers like Linear Discriminant Analysis (LDA) and Support Vector Machines (SVM) are often used because they are simple and effective in low-dimensional feature spaces. Many MI pipelines developed in the literature perform reasonably well within subjects, but they often struggle when generalizing across subjects or sessions due to differences in EEG signal characteristics [8]. This drives ongoing research into more reliable features and classifiers. Recently, deep learning methods have gained attention as alternatives to traditional feature extraction. Convolutional neural networks (CNNs) and transformer-based models have shown potential in automatically learning spatial and temporal patterns from raw EEG data. They often outperform conventional CSP-based pipelines, especially in cross-subject scenarios or when dealing with large datasets [14]. However, deep models typically need larger amounts of data to prevent overfitting, and they can be harder to interpret than classical methods. This creates trade-offs between performance and transparency.

3 Methods

A common pipeline structure, as documented in previous studies, was followed [6], [8]

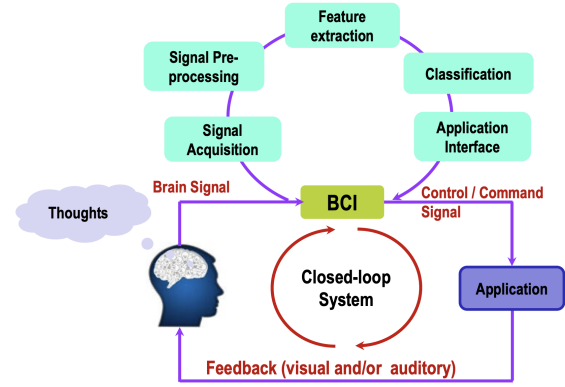


Figure 1: Common BCI pipeline [15]

The steps shown in Figure 1 can be summarized as follows:

1. **Pre-processing (3.2, 3.2):** This is a standard step used to clean the raw signal from noise and to restrict it to the frequency bands of interest, namely mu and beta [8], [9]. These operations do not account for spatial information, that is, the channel dimension.
2. **Feature extraction (3.3):** The purpose of this step is to process the data and project it into a different space where most of the relevant information is preserved, while making it more efficient for the models being used [13]. Since this process is not an exact science, scalability and flexibility were enforced through the use of git, Python notebooks, and multiple files, allowing different methods to be tested and exchanged in an interchangeable way.
3. **Classification (3.4):** After obtaining a data structure of the form (samples, features), it is fed to a classifier, which can be either supervised, as in our case, or unsupervised [7].
 - (a) *Supervised:* the model iterates over the samples, analyzes the features to predict a class, and uses the prediction error to self-correct.
 - (b) *Unsupervised:* the model explores the structure of the data without access to

labels, grouping or organizing samples based on intrinsic similarities in the feature space.

Among supervised approaches, models can be further divided into ML-based and DL-based:

- (a) *ML-based*: these rely strongly on the pre-processing and feature extraction steps described above [7].
- (b) *DL-based*: these rely on configurations that are learned directly from the network architecture [14].

The next sections we present the different methods used for each of this steps. All methods have been implemented in the [16].

3.1 Static filtering

This family of filters can be used to perform basic signal transformations on a per-channel basis. The choice of which ones to use strongly depends on the task, since for example certain frequencies may or may not be removed based on the stimuli response [8].

3.1.1 Notch filter

A second-order IIR notch filter removes 50 Hz power-line noise. The quality factor $Q = 30$ controls bandwidth via $BW = f_0/Q$, creating a narrow stopband that targets mains interference while preserving adjacent frequencies.

We used the `scipy.signal.iirnotch` library function as implementation for this method.

3.1.2 Chebyshev Type II bandpass filter

A 12th-order ($N = 6$) Chebyshev Type II bandpass IIR filter isolates 8-30 Hz. Type II design equiripples the stopband for sharp attenuation (40 dB minimum) beyond cutoffs, with monotonic passband response ideal for EEG rhythm preservation [9].

We used the `scipy.signal.cheby2` library function as implementation for this method.

3.2 Spatial filtering

This family of filters can be used to learn the spatial structure of the data. Once this is learned, they provide tools to project the data into a lower-dimensional space, resulting in a smaller magnitude for each data-point while preserving the maximum original information [8], [10]. In practice, they produce a transformation matrix whose picked K components can be used to project the M dimensional data in a ($K < M$) dimensional space. The choice of such components depends on the specific filter being used which, in turn, depends on the task. Unlike static filters, they explicitly account for the *spatial* dimension of the signal, namely the channels.

Analogously to the mentioned categorization for classification algorithms of section 3, methods from this family can be separated in *supervised* and *unsupervised* as well.

3.2.1 Common Spatial Pattern

Common Spatial Pattern (CSP) is a **supervised** spatial filter used to project high-dimensional multichannel data into a lower-dimensional space while retaining the information useful for classification [10]. It is considered supervised as the model reasons about the labels associated to the said data.

In our case, CSP aims to find a linear transformation that emphasizes differences in **variance between two classes**, making the projected data more discriminative [10]. Assuming a multichannel signal of $\mathbf{X}_i \in \mathbb{R}^{C \times T}$, where C is the number of channels and T the number of samples, belonging to class $i \in \{1, 2\}$. For each class, we compute the *normalized spatial covariance matrix*:

$$\Sigma_i = \frac{\mathbf{X}_i \mathbf{X}_i^\top}{\text{trace}(\mathbf{X}_i \mathbf{X}_i^\top)}.$$

CSP seeks a projection matrix $\mathbf{W} \in \mathbb{R}^{C \times C}$ such

that the projected signals $\mathbf{Z} = \mathbf{W}^\top \mathbf{X}$ maximize the variance for one class while minimizing it for the other [10]. This is achieved by solving the generalized eigenvalue problem

$$\Sigma_1 \mathbf{w} = \lambda (\Sigma_1 + \Sigma_2) \mathbf{w}.$$

The eigenvectors $\mathbf{w}$ corresponding to the largest and smallest eigenvalues define the said spatial filters. They optimally separate the two classes in terms of variance. By *projecting* the data onto a subset of these filters ($m = 3$ from both ends of the matrix), we effectively split the original high-dimensional dataset into optimal directions for classification.

3.2.2 Principal Component Analysis

Principal Component Analysis (PCA) is an **unsupervised** spatial filtering technique used to project high-dimensional multichannel data into a lower-dimensional space while preserving as much of the original **variance** as possible [8]. Unlike CSP, it does not rely on class labels and instead considers the dataset as a whole.

Assuming a multichannel signal $\mathbf{X} \in \mathbb{R}^{C \times T}$, where C is the number of channels and T the number of samples, we first compute the *spatial covariance matrix*:

$$\Sigma = \frac{1}{T} \mathbf{X} \mathbf{X}^\top.$$

PCA seeks a projection matrix $\mathbf{W} \in \mathbb{R}^{C \times C}$ such that the projected signals $\mathbf{Z} = \mathbf{W}^\top \mathbf{X}$ have maximal variance along the first components, with each subsequent component capturing the largest remaining variance while being orthogonal to the previous ones. This is obtained by solving the eigenvalue problem

$$\Sigma \mathbf{w} = \lambda \mathbf{w}.$$

The eigenvectors $\mathbf{w}$ associated with the largest eigenvalues define the principal components, which act as spatial filters capturing the most informative directions of variance in the data [8]. By *projecting* the original signals onto a subset of these components

($k = 6$ components), we reduce the dimensionality of the dataset while retaining the main sources of information (at least 95 percent of the variance).

We used the `sklearn.decomposition.PCA` library function as implementation for this method.

3.3 Feature extraction

Although the dataset already contains structured features, such as PCA components over time samples, an additional transformation is required to further compress the information they carry. This step is motivated by the fact that ML-based methods mentioned in section 3 are generally not able to directly learn the full temporal structure of the data and instead require a compact representation that summarizes the most relevant characteristics of the signal [13]. For this reason, we apply feature extraction techniques that collapse the temporal dimension into more informative features.

3.3.1 Short-Time Fourier Transform

The Short-Time Fourier Transform (STFT) is a time-frequency analysis technique used to characterize the spectral content of non-stationary signals (EEG in this case) over time [8]. Unlike the classical Fourier Transform, which provides a global frequency representation, STFT allows us to analyze how the frequency components of a signal evolve within small temporal windows.

Given a discrete-time signal $x[n]$, the Short-Time Fourier Transform (STFT) is defined as

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] w[n - m] e^{-j\omega n},$$

where $w[n]$ is a window function centered at time index m and ω denotes the angular frequency. By sliding the window across the signal, the STFT produces a two-dimensional representation that jointly captures temporal and spectral information. In our implementation, the transform is applied along the

temporal axis using the known sampling frequency, resulting in a set of frequency bins for each time window and spatial component.

In practice, we work with the magnitude of the transform,

$$M(m, \omega) = |X(m, \omega)|,$$

which is used as the feature representation. The magnitude spectrum provides a direct measure of how signal energy is distributed across the different frequencies. To obtain a time-independent description, the magnitude is averaged across the timesteps dimension.

Finally, the frequency dimension is condensed by summing the log-scaled (for numerical stability) magnitude spectrum over the mu and beta bands. Through this process, the time series is mapped into a lower-dimensional space that preserves frequency-related information for the bands of interest.

We used the `scipy.signal.stft` library function as implementation for this method.

3.4 Classification

3.4.1 Feedforward Neural Network

The Feedforward Neural Network (FNN) is a **supervised**, **ML-based** classification model that operates on **fixed-size** feature vectors. In our case, the network takes as input the features extracted during the feature extraction stage (3.3). The network consists of a sequence of fully connected layers that learn a nonlinear mapping between the input features and the corresponding class labels. Since the model does not explicitly model spatial or temporal structure, its performance strongly depends on the quality and informativeness of the extracted features, as well as the hyperparameters defining the network.

A Feedforward Neural Network maps the input feature vector $\mathbf{x} \in \mathbb{R}^d$ to an output vector $\mathbf{y} \in \mathbb{R}^C$ representing class scores, where d is the number of features and C the number of classes. Each layer per-

forms a linear transformation followed by a nonlinear mapping:

$$\mathbf{h}^{(l)} = g(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\mathbf{h}^{(0)} = \mathbf{x}$, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases of layer l , and $g(\cdot)$ is a generic nonlinear function. The final output layer projects the last hidden representation to the output space:

$$\mathbf{y} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}.$$

After experimenting with different network architectures, the resulting configuration which achieved the best performance is the following:

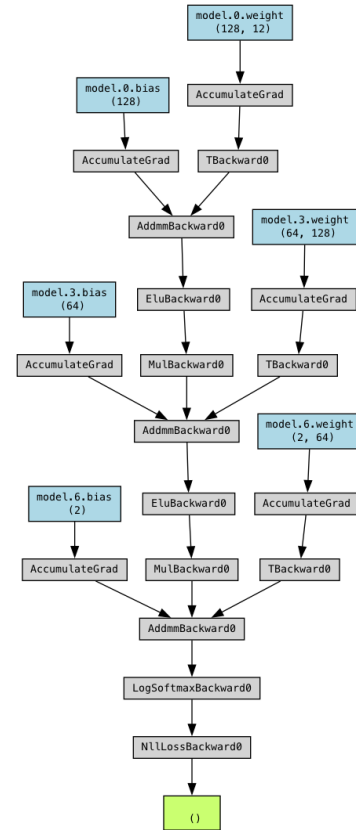


Figure 2: FNN architecture which achieved the best performance [16]

The presented configuration first expands the feature space and then progressively reduces it. Dropout layers are applied after each layer to limit the overfitting. The ELU activation function is used, as it is commonly adopted in EEG-related applications.

We used the `torch` library for the implementation of this method.

3.4.2 Convolutional Neural Network

The Convolutional Neural Network (CNN) is a **supervised, DL-based** classification model that operates directly on the time-series data. Unlike the FNN, the CNN does not rely on pre-extracted features (3.3) but uses the raw time samples, allowing it to capture temporal patterns and local dependencies through shared convolutional weights. This makes the model effective at learning from the full temporal structure of the signal.

Let the input signal be $\mathbf{X} \in \mathbb{R}^{C \times T}$, where C is the number of channels and T the number of time samples. Each convolutional layer applies a set of filters $\mathbf{K}_k \in \mathbb{R}^{C \times F}$ across the temporal dimension to produce feature maps:

$$\mathbf{Z}_k[t] = g \left(\sum_{c=1}^C \sum_{f=0}^{F-1} \mathbf{K}_k[c, f] \cdot \mathbf{X}[c, t + f] + b_k \right),$$

where F is the filter length, k indexes the filters, b_k is the bias term, and $g(\cdot)$ is a generic nonlinear mapping. The resulting feature maps are optionally downsampled via pooling layers and eventually flattened into a vector that is fed to fully connected layers:

$$\mathbf{y} = \mathbf{W}_{\text{fc}} \text{vec}(\mathbf{Z}) + \mathbf{b}_{\text{fc}},$$

producing the class scores.

After experimenting with different network architectures, the resulting configuration which achieved the best performance is presented in Figure 9. The referenced CNN architecture progressively increases the number of feature maps while reducing the temporal resolution of the signal. Initial convolutional lay-

ers with large kernels capture local temporal patterns across channels, while max-pooling downsamples the time dimension progressively. Batch normalization is used to stabilize training, and ELU activations are adopted due to their effectiveness in EEG-related tasks. Dropout layers are inserted after both convolutional and fully connected stages to limit overfitting. As usual, a flattening operation is applied allowing the network to continue with fully-connected layers, which are then mapped to the same space for classification as in the other case.

We used the `torch` library for the implementation of this method.

3.4.3 Cross-validation

Several cross-validation strategies were employed, including *stratified k-fold*, *leave-one-group-out*, and *train-test split*. These approaches are used to obtain more reliable performance estimates, reduce bias in parameter learning, and ensure that the evaluation is not biased by a specific data partition.

It consists on training and testing the model on different and randomly sampled subsets of the data, providing a more realistic assessment of its generalization capabilities.

We used the `sklearn.model_selection` library for the implementation of these methods.

3.4.4 Evaluation

Considering the problem as a classification task and including the cross-validation strategy mentioned above, the different models were evaluated using *accuracy* and *cross-entropy loss*. The performance metrics were averaged across folds or batches, depending on the adopted validation strategy.

4 Results

In this chapter the results of the used [3](#) methods are step by step illustrated and visually presented.

4.1 Static filtering

The following figure presents a comparison of the raw EEG signal, a notch-filtered EEG signal, a band-pass filtered EEG signal, and an EEG signal processed with both filters.

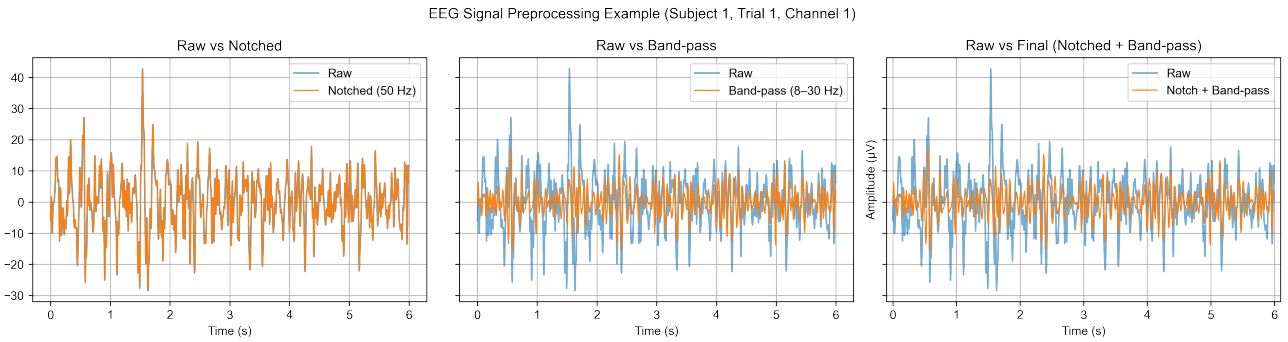


Figure 3: EEG signal filtered with notch and band-pass filters. Left: notch filter. Middle: band-pass filter. Right: combined filters.

As shown in the left panel, the notch filter has no effect, as the notch-filtered signal overlaps with the raw EEG. The middle panel illustrates the effect of the band-pass filter, which narrows the frequency content of the signal. In the right panel, the combined application of notch and band-pass filters produces a signal progression similar to that of the band-pass filtered signal alone.

4.2 STFT

The next figure shows how the previously shown signal changes before and after the STFT application, resulting in the transformed signal in the time–frequency domain. This representation allows the observation of frequency components over time, which is required for further analysis of motor imagery–related activity.

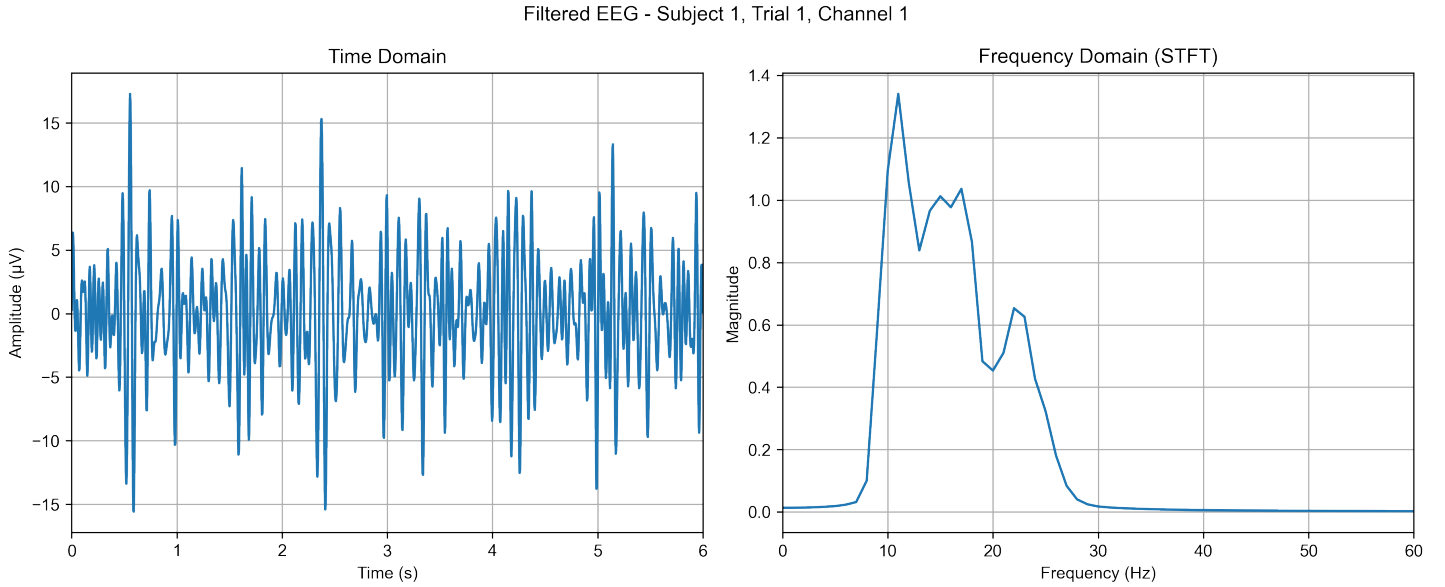


Figure 4: Filtered EEG signal and its time–frequency representation obtained via STFT.

4.3 PCA

The PCA-filtered signal is shown in the next figure. It is worth noting that the visual appearance is very similar to the previous one. This is expected, since PCA reduces the channel dimension, while leaving the remaining dimensions of the signal unchanged.

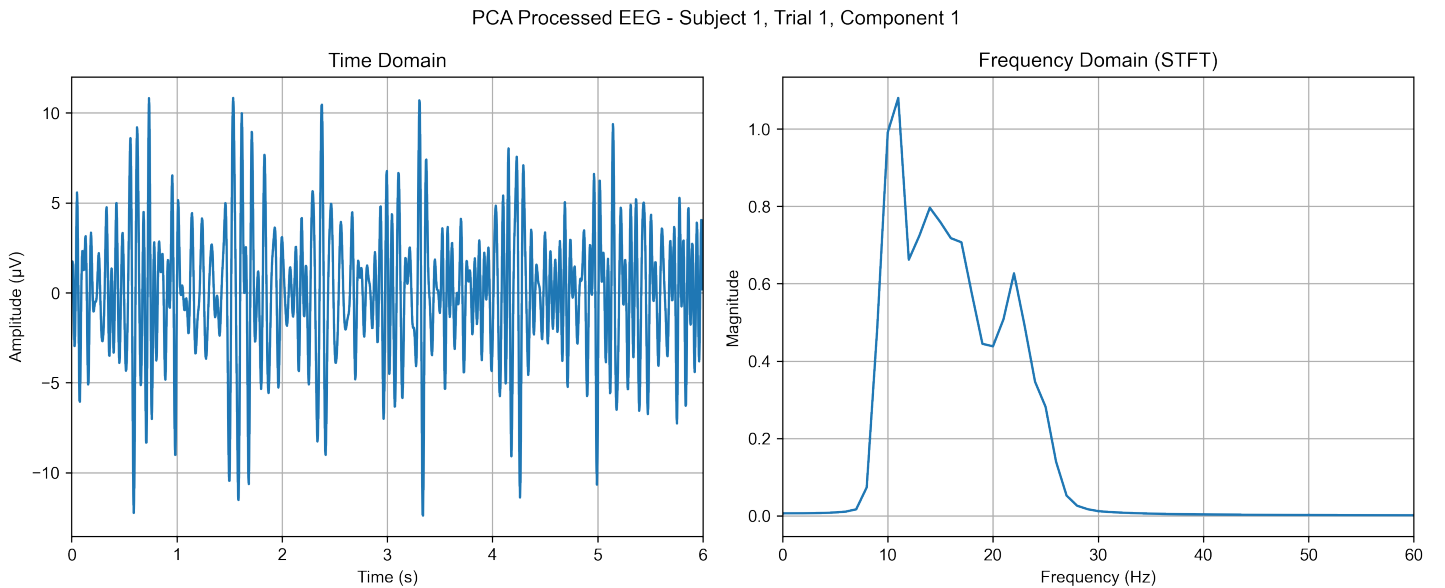


Figure 5: PCA-processed EEG signal.

The number of components shown in the previous figure corresponds to $k = 3$ principal components. This value was determined by inspecting the explained variance associated with each component and selecting the minimum number of components required to reach a cumulative explained variance of 95%.

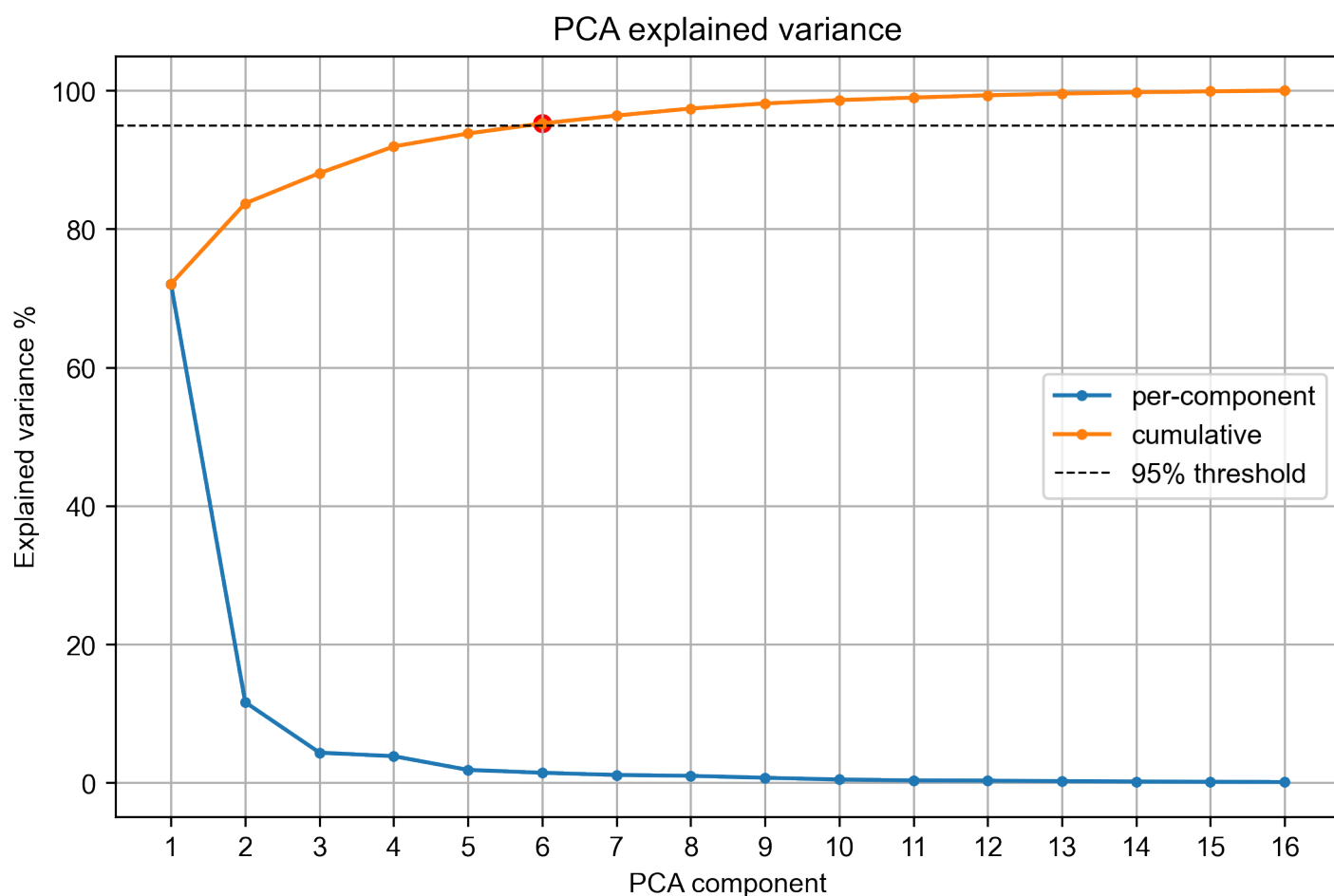


Figure 6: PCA variance in relation to the number of components retained.

4.4 CSP

The CSP-filtered signal is shown in the figure. Similarly to the PCA case, the signal does not look very different from the original one. This is expected, since also CSP acts on the channel dimension by recombining sensors into spatial components, while the temporal evolution of the signal is kept.

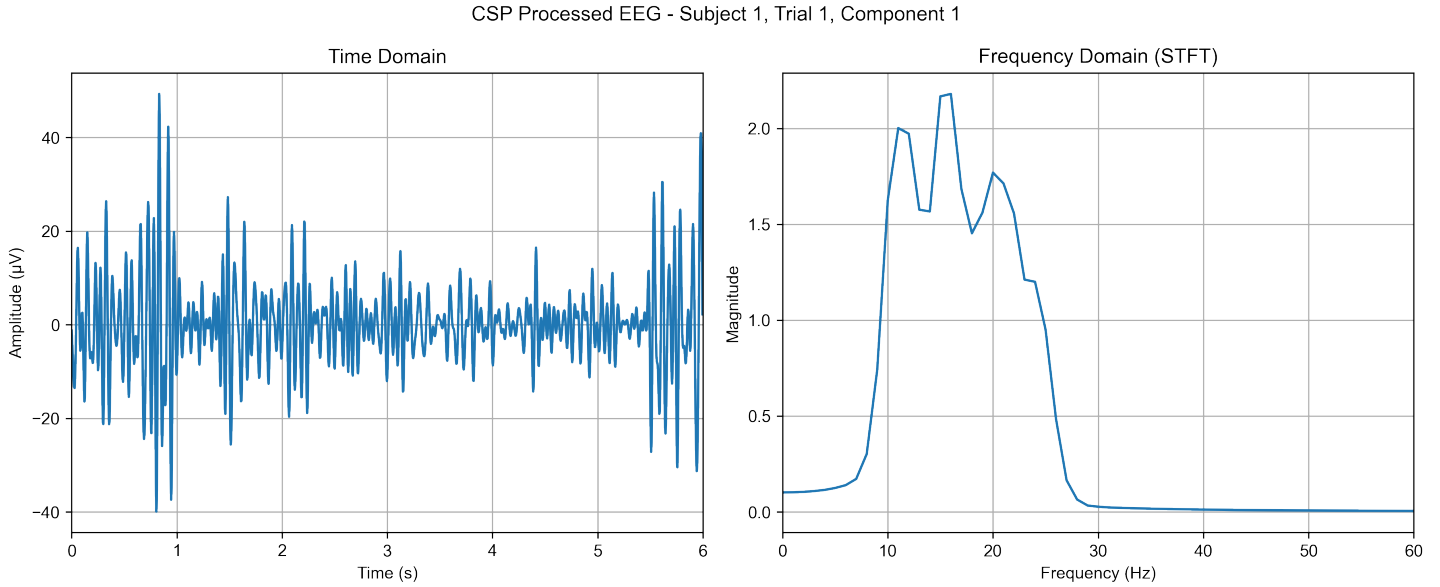


Figure 7: CSP-processed EEG signal.

4.5 Neural networks (FNN and CNN)

The next figure illustrates two types of neural networks compared in terms of training and test performance. In (a), the FNN trained with 1000 epochs is shown. The validation loss decreases exponentially with increasing epochs. The validation accuracy increases rapidly from 55% to 73% within the first 100 epochs, followed by a slower and nearly constant increase, reaching a maximum of approximately 74%. In (b), the same two plots are shown using a CNN as classifier. The validation loss starts at around 75% and remains nearly constant over the entire training with 30 epochs. The validation accuracy fluctuates, forming a steadily increasing sawtooth pattern, and reaches a maximum of 75%.

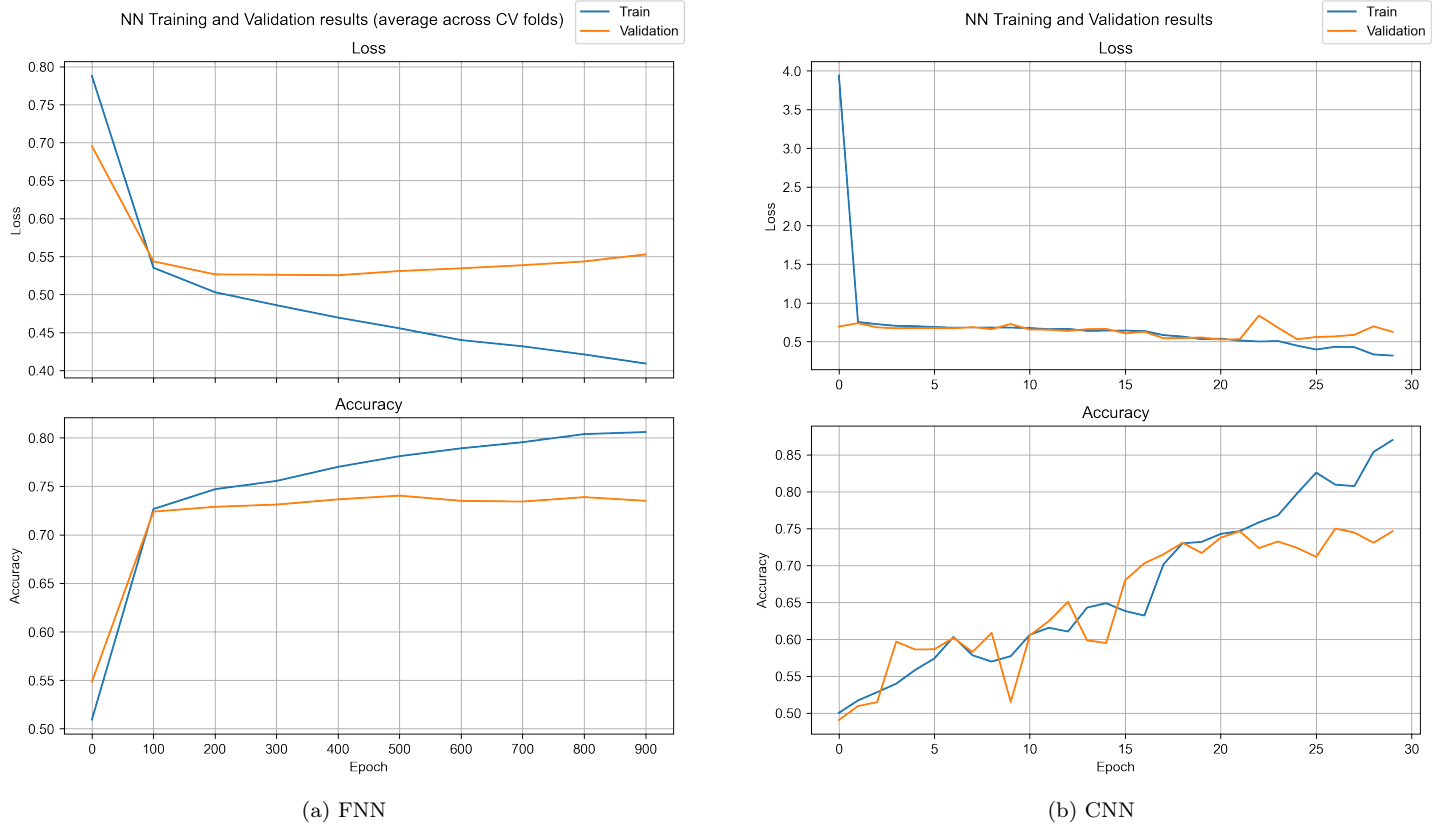


Figure 8: Comparison of the FNN and CNN architectures based on their training and test performance.

4.6 Dimensionality across the pipeline

A summary of the evolution of data dimensionality across the entire processing pipeline described in Chapter 3 is presented. For each step, both the numerical dimensions and their semantic meaning are reported, highlighting how the data are progressively transformed from raw multichannel time-series to a compact representation used for classification.

No.	Step	Dimensions	Shape
1	Raw data	$(6, 434^1, 1536, 16)$	(subjects, trials, timepoints, channels)
2	Notch + Band-pass filter (3.2)	$(6, 434, 1536, 16)$	(subjects, trials, timepoints, channels)
3	CSP filter (3.2.1)	$(6, 434, 1536, 6)$	(subjects, trials, timepoints, components)
4	STFT (3.3.1)	$(6, 434, 129, 6)$	(subjects, trials, frequencies, components)
5	Band average magnitude (3.3)	$(6, 434, 2^2, 6)$	(subjects, trials, bands, components)
6	FNN (3.4.1) IN	$(2604^3, 12)$	(subjects x trials, components x bands)
7	FNN (3.4.1) OUT	(2604)	(subjects x trials ⁴)

Table 1: Dimensionality *after* each pipeline step in Chapter 3

5 Discussion

5.1 Pre-processing

The pre-processing step aimed to clean the EEG signal and focus on the mu and beta bands, which are most relevant for motor imagery. The notch filter targets 50 Hz power-line noise, but in this dataset it had little visible effect (Figure 1). The likely reason is that the data were already pre-filtered, so the notch filter was not strictly necessary. Still, it was applied to follow standard signal processing practices.

The band-pass filter, in contrast, clearly shapes the signal by restricting its frequency content to the bands of interest. Applying both filters together does not produce a markedly different result from the band-pass alone, though the notch filter ensures that any residual line noise is removed. Overall, these static filters effectively remove irrelevant frequency components without distorting the informative signal. One limitation is that they treat each channel independently and do not account for spatial correlations, so some structured noise might remain.

5.2 Feature Extraction

Feature extraction relied on spatial filtering using PCA and CSP. PCA reduces dimensionality while preserving the majority of variance in the data (Figure 6). Keeping six principal components captures 95% of the variance, resulting in a more compact dataset that is easier to handle computationally. The drawback is that PCA ignores class labels, so it does not optimize for distinguishing between left- and right-hand imagery.

CSP, on the other hand, is supervised and maximizes variance differences between the classes. When combined with STFT, the CSP-filtered signals show

a more “spiky” time–frequency pattern (Figure 7), which reflects stronger discriminative features. CSP benefits from using label information, but it is sensitive to noise and outliers and requires enough labeled data to compute stable filters.

STFT provides a time–frequency view of the signals, showing how spectral content evolves over time (Figure 4). This is important for motor imagery because mu and beta rhythms can vary in timing and intensity. Averaging the magnitude across these bands reduces the feature dimensionality and creates a fixed-size input suitable for machine learning models. The trade-off is that fine temporal details may be lost, potentially omitting brief but informative signal patterns.

5.3 Classification

Two neural network architectures were tested: FNNs and CNNs (Figure 8). The FNN relies on pre-extracted features and achieved a validation accuracy of about 75%, with smooth convergence in the loss. This shows that a well-prepared feature set can capture most of the information needed for classifying motor imagery. Its advantages include simplicity, interpretability, and lower computational demand compared to deep networks. The main limitation is that it cannot exploit temporal or spatial structure beyond what the features already provide.

The CNN operates directly on raw EEG, learning temporal patterns automatically. Its peak accuracy was similar (73%), but the validation accuracy fluctuated in a sawtooth pattern, and the loss remained nearly constant. This suggests that CNNs are more sensitive to hyperparameters and the amount of training data. CNNs can potentially capture richer signal patterns, but they require more data and are harder to interpret compared to classical ML models.

¹No. of trials was different between the subjects, therefore we downsampled along the trial dimension to the minimum trial count.

²Only the mu and beta bands are kept.

³Samples were further split in K-folds as indicated in 3.4.3 although for brevity we presented the shape before such split.

⁴The network outputs the logits that is the likelihood of the test set items belonging to each class. This is then fed to the criterion without saving these results.

Cross-validation ensured that performance estimates were reliable and generalizable across different data splits. Overall, the results highlight the importance of combining informative features with a model appropriate for the dataset size and structure.

6 Conclusions

The final pipeline combines notch + band-pass filtering, CSP spatial filtering, STFT with band-averaged magnitude, and FNN classification.

- Notch + band-pass filters: Remove line noise and focus the signal on relevant frequency bands without introducing distortions.

- CSP: Creates features with strong class-specific discriminative power for left- versus right-hand imagery.

- STFT + band-averaged magnitude: Captures the temporal evolution of mu and beta rhythms while producing fixed-size features for the FNN.

- FNN: Balances simplicity, interpretability, and high classification accuracy. It avoids overfitting while leveraging the discriminative information provided by CSP and STFT.

This combination achieves good performance while keeping the processing pipeline interpretable and computationally efficient. Compared to PCA or CNN-based alternatives, it offers a robust approach that emphasizes class-specific features, which is especially important for small datasets with few subjects or trials.

There are several ways to further improve the pipeline. Adaptive or filter bank CSP could better capture subject-specific frequency variations. Hybrid models that combine CNNs with CSP features could take advantage of both feature-based and deep-learning approaches. Artifact removal or spatially-aware noise suppression could increase robustness. Expanding the dataset with more subjects or trials would allow exploration of more complex deep learning models without overfitting.

A Appendix

A.1 CNN architecture

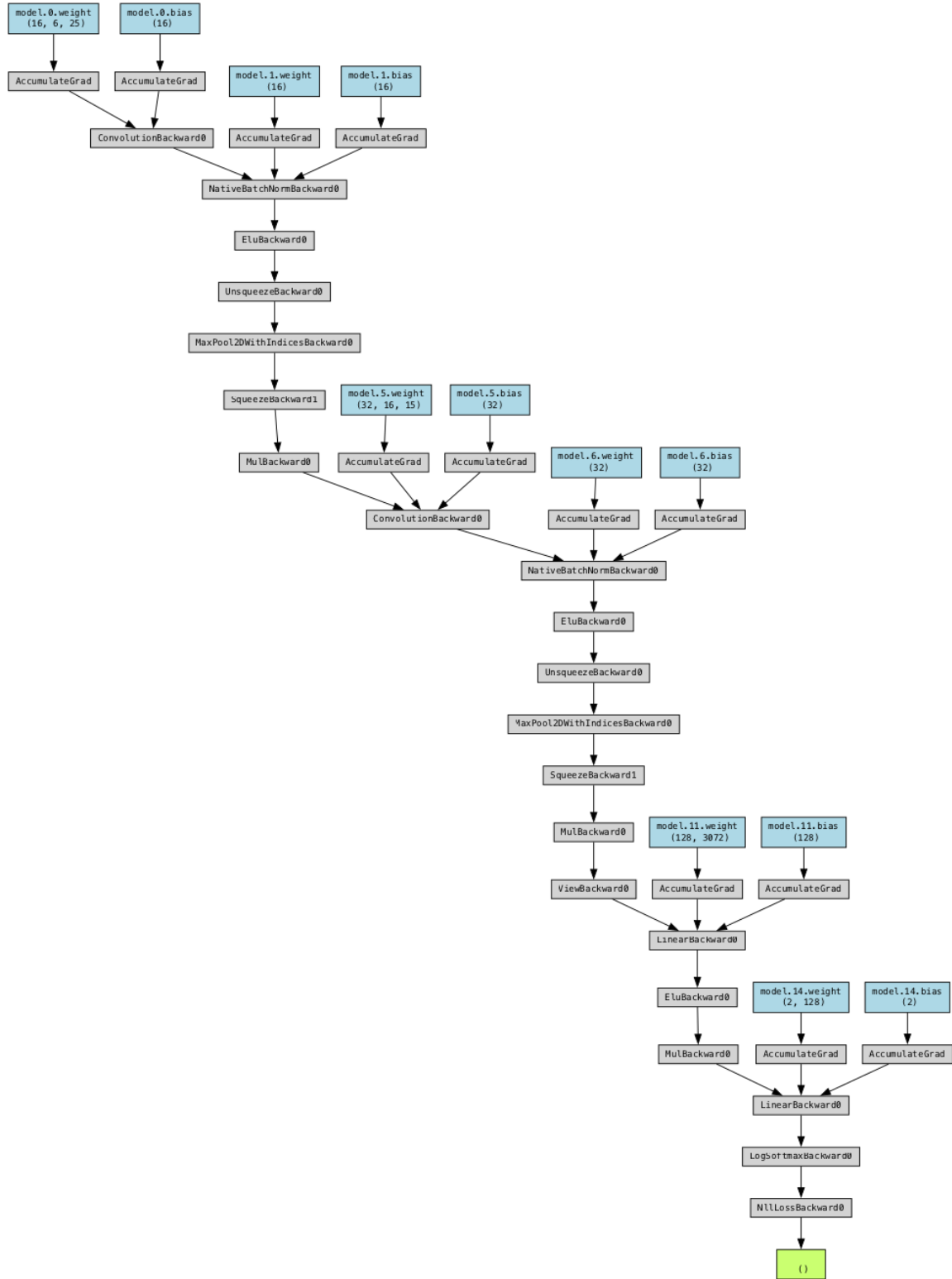


Figure 9: CNN architecture which achieved the balance between performance and computational cost [16]

References

- [1] A. Batista, “Chapter 17 - Brain-computer interfaces for basic neuroscience,” en-US, in *Handbook of Clinical Neurology*, vol. 168, Elsevier, Jan. 2020, pp. 233–247. DOI: [10.1016/B978-0-444-63934-9.00017-2](https://doi.org/10.1016/B978-0-444-63934-9.00017-2). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.sciencedirect.com/science/chapter/handbook/pii/B9780444639349000172>.
- [2] M. v. S. Umcu et al., “Title: Contribution to Roadmap,” 2014. Accessed: Jan. 20, 2026. [Online]. Available: <https://www.semanticscholar.org/paper/Title%3A-Contribution-to-Roadmap-Umcu-Umcu/5cb411de3db4941d5c7ecfc19de8af9243fb63d5?p2df>.
- [3] L. Ferrero, M. Ortiz, V. Quiles, E. Iáñez, J. A. Flores, and J. M. Azorín, “Brain Symmetry Analysis during the Use of a BCI Based on Motor Imagery for the Control of a Lower-Limb Exoskeleton,” en, *Symmetry*, vol. 13, no. 9, Sep. 2021, ISSN: 2073-8994. DOI: [10.3390/sym13091746](https://doi.org/10.3390/sym13091746). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.mdpi.com/2073-8994/13/9/1746>.
- [4] A. Nijholt, “The future of brain-computer interfacing (keynote paper),” in *2016 5th International Conference on Informatics, Electronics and Vision (ICIEV)*, May 2016, pp. 156–161. DOI: [10.1109/ICIEV.2016.7759987](https://doi.org/10.1109/ICIEV.2016.7759987). Accessed: Jan. 20, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7759987>.
- [5] V. Mondini, A. L. Mangia, and A. Cappello, “EEG-Based BCI System Using Adaptive Features Extraction and Classification Procedures,” en, *Computational Intelligence and Neuroscience*, vol. 2016, no. 1, p. 4562601, 2016, _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2016/4562601>, ISSN: 1687-5273. DOI: [10.1155/2016/4562601](https://doi.org/10.1155/2016/4562601). Accessed: Jan. 20, 2026. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/4562601>.
- [6] A. Saibene, M. Caglioni, S. Corchs, and F. Gasparini, “EEG-Based BCIs on Motor Imagery Paradigm Using Wearable Technologies: A Systematic Review,” en, *Sensors*, vol. 23, no. 5, Mar. 2023, ISSN: 1424-8220. DOI: [10.3390/s23052798](https://doi.org/10.3390/s23052798). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.mdpi.com/1424-8220/23/5/2798>.
- [7] R. Abiri, S. Borhani, E. W. Sellers, Y. Jiang, and X. Zhao, “A comprehensive review of EEG-based brain-computer interface paradigms,” en, *Journal of Neural Engineering*, vol. 16, no. 1, p. 011001, Jan. 2019, ISSN: 1741-2552. DOI: [10.1088/1741-2552/aaf12e](https://doi.org/10.1088/1741-2552/aaf12e). Accessed: Jan. 20, 2026. [Online]. Available: <https://doi.org/10.1088/1741-2552/aaf12e>.
- [8] N. Padfield, J. Zabalza, H. Zhao, V. Masero, and J. Ren, “EEG-Based Brain-Computer Interfaces Using Motor-Imagery: Techniques and Challenges,” en, *Sensors*, vol. 19, no. 6, Mar. 2019, ISSN: 1424-8220. DOI: [10.3390/s19061423](https://doi.org/10.3390/s19061423). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.mdpi.com/1424-8220/19/6/1423>.
- [9] H. Yuan and B. He, “Brain-Computer Interfaces Using Sensorimotor Rhythms: Current State and Future Perspectives,” *IEEE transactions on bio-medical engineering*, vol. 61, no. 5, pp. 1425–1435, May 2014, ISSN: 0018-9294. DOI: [10.1109/TBME.2014.2312397](https://doi.org/10.1109/TBME.2014.2312397). Accessed: Jan. 20, 2026. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4082720/>.

- [10] Z. Ma, K. Wang, M. Xu, W. Yi, F. Xu, and D. Ming, “Transformed common spatial pattern for motor imagery-based brain-computer interfaces,” eng, *Frontiers in Neuroscience*, vol. 17, p. 1116721, 2023, ISSN: 1662-4548. DOI: [10.3389/fnins.2023.1116721](https://doi.org/10.3389/fnins.2023.1116721).
- [11] A. A. Shiam, K. M. Hassan, M. R. Islam, A. M. M. Almassri, H. Wagatsuma, and M. K. I. Molla, “Motor Imagery Classification Using Effective Channel Selection of Multichannel EEG,” en, *Brain Sciences*, vol. 14, no. 5, May 2024, ISSN: 2076-3425. DOI: [10.3390/brainsci14050462](https://doi.org/10.3390/brainsci14050462). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.mdpi.com/2076-3425/14/5/462>.
- [12] S. Kumar, A. Sharma, and T. Tsunoda, “An improved discriminative filter bank selection approach for motor imagery EEG signal classification using mutual information,” en, *BMC Bioinformatics*, vol. 18, no. 16, p. 545, Dec. 2017, ISSN: 1471-2105. DOI: [10.1186/s12859-017-1964-6](https://doi.org/10.1186/s12859-017-1964-6). Accessed: Jan. 20, 2026. [Online]. Available: <https://doi.org/10.1186/s12859-017-1964-6>.
- [13] R. Martín-Clemente, J. Olias, D. B. Thiyam, A. Cichocki, and S. Cruces, “Information Theoretic Approaches for Motor-Imagery BCI Systems: Review and Experimental Comparison,” eng, *Entropy (Basel, Switzerland)*, vol. 20, no. 1, p. 7, Jan. 2018, ISSN: 1099-4300. DOI: [10.3390/e20010007](https://doi.org/10.3390/e20010007).
- [14] W. Zhao, B. Zhang, H. Zhou, D. Wei, C. Huang, and Q. Lan, “Multi-scale convolutional transformer network for motor imagery brain-computer interface,” en, *Scientific Reports*, vol. 15, no. 1, p. 12935, Apr. 2025, ISSN: 2045-2322. DOI: [10.1038/s41598-025-96611-5](https://doi.org/10.1038/s41598-025-96611-5). Accessed: Jan. 20, 2026. [Online]. Available: <https://www.nature.com/articles/s41598-025-96611-5>.
- [15] S. Puthusserypady, *Lecture 1: Introduction*, Jan. 2025.
- [16] T. Balducci, *GitHub repository for MI task source code*. [Online]. Available: <https://github.com/BalducciFrancesco/22053-Motor-Imagery-Miniproject>.